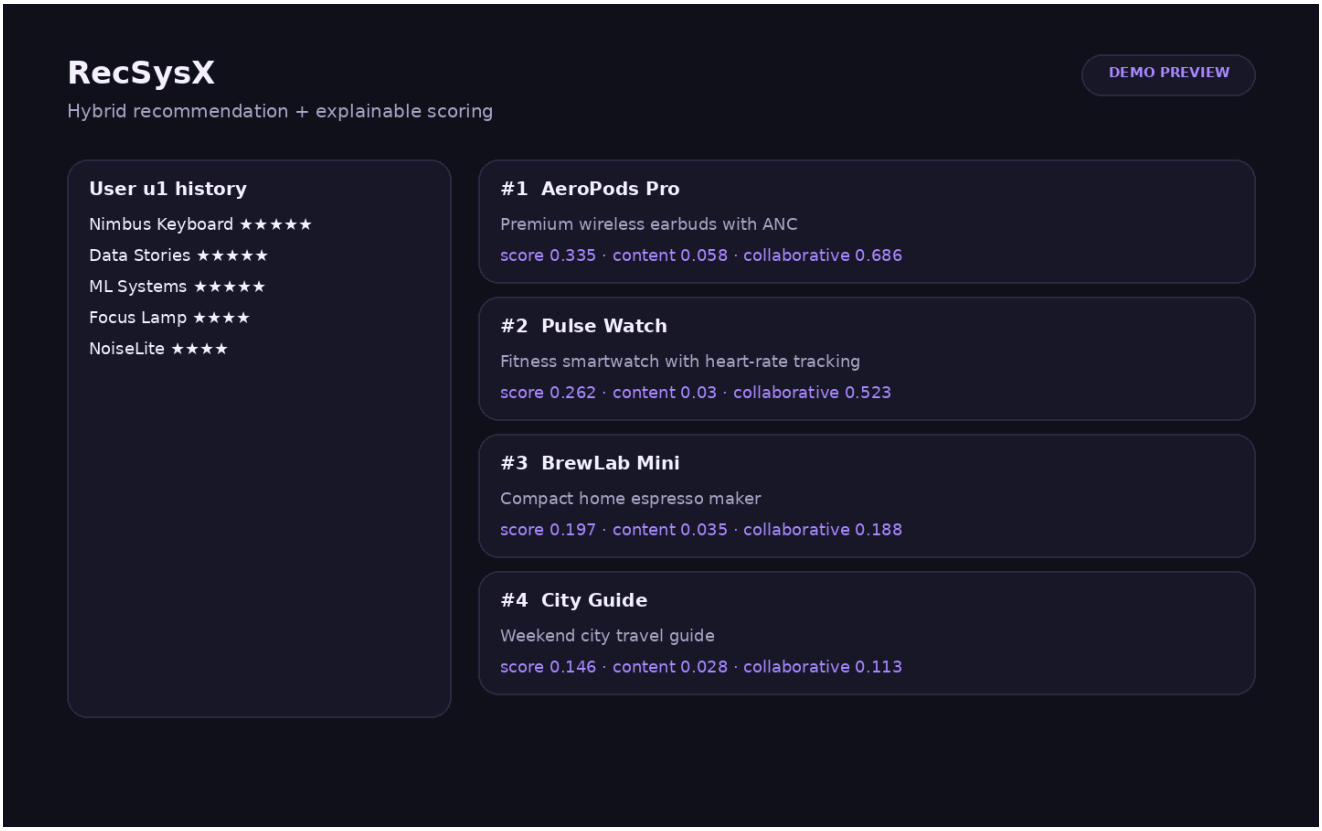
RecSysX

Hybrid Recommendation Engine

Python • Scikit-learn • Collaborative Filtering • Content Embeddings • FastAPI • PyTorch/FAISS-ready



Representative demo preview. Run the included application for the interactive version.

1. Problem and Solution

A useful recommender must personalize results even when behavioral data is sparse. RecSysX combines content similarity, collaborative user behavior and popularity while exposing score components for debugging and explainability.

Why this is strong on an AI/ML resume

The project is built as a small product rather than only a notebook. It includes a backend API, interactive frontend, repeatable local data, automated tests, Docker packaging, CI configuration, demo instructions, and clear model/engineering boundaries.

Core components

Component	Responsibility
Content Encoder	Vectorizes item category, tags and description.
User Profile	Builds a weighted content profile from rated items.
Collaborative Signal	Finds behavior similarity between users and transfers preference scores.
Hybrid Ranker	Blends signals, removes seen items and serves top-K recommendations.

2. Architecture and Data Flow

Production extension: event stream → feature store → two-tower trainer → vector index → online ranker → experimentation platform.

Engineering decisions

- Keep the core demo fully local so it is reliable during interviews.
- Separate model logic from API/UI so algorithms can be upgraded without rewriting the product surface.
- Use explicit validation and automated tests so the demo can be explained as an engineering system, not only an ML experiment.
- Package with Docker and CI configuration to demonstrate repeatability.

What “production-style” means here

The repositories demonstrate production patterns, but they are portfolio systems, not claims of enterprise production deployment. A real deployment would add identity/access management, secret management, persistent observability, load testing, threat modeling, data governance, backup/rollback processes and infrastructure-specific scaling.

3. Live Demo Walkthrough

1. Select user u1 and show interaction history.

2. Explain why already-seen items are removed.

3. Walk through content and collaborative score components for a recommendation.

4. Switch users and show personalized changes.

5. Explain scale-up using learned two-tower embeddings plus an ANN index such as FAISS.

What to say while demoing

> *RecSysX solves personalization using a hybrid ranking strategy. Item metadata is converted into TF-IDF content vectors, while the user-item interaction matrix provides collaborative signals. For each user I build a content profile, estimate collaborative preference from similar users, blend those signals with a small popularity prior, remove items the user already consumed, and return top-K results. The dashboard exposes the score components, which is useful for debugging and explainability.*

Demo failure fallback

Keep `demo_output.json` and the preview image in the repository. If a live environment fails, walk through the saved output, explain the expected API call, then show the unit tests. This demonstrates preparedness without pretending the failed live run succeeded.

4. Resume Bullets

- Built a hybrid recommendation engine combining content embeddings, collaborative user behavior and popularity signals to generate personalized top-K product recommendations.
- Implemented score explainability, cold-start-friendly content features, offline catalog-coverage metrics, FastAPI model serving, Docker deployment and automated recommendation tests.
- Added optional PyTorch two-tower embedding and FAISS ANN modules for scaling the same API toward learned retrieval and low-latency vector search.

Important claim rule

Only claim technologies you can run and explain. Optional modules such as YOLO, PyTorch two-tower models, FAISS, MLflow or Airflow are included as upgrade paths where noted; do not say you trained/deployed them until you actually execute that path and record evidence/metrics.

Metrics to discuss

Precision@K	Recall@K	NDCG@K	Hit Rate@K	Catalog coverage / diversity

5. Interview Questions

How do you evaluate?

Precision@K, Recall@K, NDCG@K, hit rate, coverage, diversity, novelty and online CTR/conversion.

How would you serve millions of items?

Learn user/item embeddings, precompute item vectors, use FAISS/HNSW ANN retrieval, then apply a smaller ranking model to a few hundred candidates.

Questions you should be ready for

- Why did you choose this algorithm instead of an alternative?
- What is the biggest failure mode in the current implementation?
- How would the architecture change at 100x traffic/data size?
- Which offline and online metrics would you monitor?
- What did you personally implement, test and debug?

6. Upgrade Roadmap

- Add authentication and role-based access.
- Persist model/retrieval artifacts with explicit versioning.
- Add structured observability: request logs, latency, error rates and model metrics.
- Create a small evaluation suite with expected outputs and regression thresholds.
- Deploy to a cloud service and record the public demo URL in the README.
- Add one real dataset experiment and document the training/evaluation results.

Final presentation advice

Do not memorize only the feature list. Be able to draw the data flow, explain one algorithm choice, show one failure case, quote one meaningful metric and describe one scalability improvement. That is what turns the project from a resume line into an interview asset.